

Movie Ticket Booking System (CLI)

A command-line application specification and object-oriented implementation guide.

Console application · Python · Object-Oriented Programming

This document specifies a command-line **Movie Ticket Booking System** intended as a practical exercise in object-oriented design. Rather than organising the program as a collection of standalone functions, the application is structured around well-defined classes that model the problem domain, encapsulate their own state, and interact through clearly defined interfaces. The goal is to practise real-world object-oriented thinking: identifying entities, assigning responsibilities, and protecting internal data.

The completed application allows a user to view available movies, view seat availability for a chosen movie, book a seat, cancel a booking, and display the current booking status.

1. Objective

Build a menu-driven command-line booking system using object-oriented concepts. The user interacts with a numbered text menu and performs the following operations:

- View available movies
- View available seats for a selected movie
- Book a ticket for a specific seat
- Cancel an existing booking
- Display the current booking status

2. Learning Outcomes

On completion, the developer should be able to demonstrate the following competencies:

- **Class creation** — defining classes that represent real entities in the problem domain.
- **Object creation** — instantiating those classes and managing multiple objects.
- **Constructors** — initialising object state through `__init__`.
- **Encapsulation** — hiding internal data and exposing it only through controlled methods.
- **Object interaction** — having one object delegate work to another rather than manipulating its data directly.
- **Lists and dictionaries** — choosing appropriate built-in collections to store seats and movies.
- **Basic validation** — guarding operations against invalid or conflicting input.
- **Menu-driven application design** — organising the program around a repeating user-input loop.

3. Functional Requirements

3.1 Display movies

The user can list every movie currently available in the system. Each movie is presented with a selection number.

```
Available Movies
```

- ```
1. Leo
2. Jailer
3. Avengers
```

### 3.2 View available seats

After selecting a movie, the user sees only the seats that are still free for that movie.

```
Movie: Leo
```

```
Available Seats:
A1 A2 A3 A4 A5
```

### 3.3 Book a ticket

The user selects a movie and enters a seat number. The system first checks whether the seat exists and is available; if so, it reserves the seat. If the seat is already taken, the booking is rejected with a clear message.

```
Enter seat number: A2
```

```
Ticket booked successfully
```

When the requested seat is unavailable:

```
Seat already booked
```

### 3.4 Cancel a ticket

The user enters the seat number to release. The system verifies the seat is currently booked, then returns it to the available pool.

```
Enter seat number to cancel: A2
```

```
Booking cancelled successfully
```

### 3.5 View booking status

The system reports both the booked and the available seats for a movie.

Booked Seats:

A2 A4

Available Seats:

A1 A3 A5

## 4. Object-Oriented Design

The system is built from two classes. The `Movie` class models a single film and owns its seat data; the `BookingSystem` class manages the overall application and the collection of movies. Responsibilities are kept separate so that each class has a single, clear purpose.

### Class: `Movie`

Represents one movie and the state of its seats. This class is responsible for everything related to its own seats — and nothing else.

| Aspect              | Description                                                                                                                                                                         |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsibility      | Store the movie name and seat data; book a seat; cancel a seat; report available and booked seats.                                                                                  |
| State (attributes)  | The movie name, the collection of available seats, and the collection of booked seats. The booked-seat data is kept private.                                                        |
| Behaviour (methods) | A method to book a seat, a method to cancel a seat, a method to show available seats, and a method to show booked seats. A read-only accessor exposes the private booked-seat data. |

### Class: `BookingSystem`

Acts as the controller for the whole application. It holds all `Movie` objects and drives the user interface, but it never manipulates a movie's seat data directly — it always asks the relevant `Movie` object to perform the operation.

| Aspect             | Description                                                                                                                                       |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsibility     | Manage the set of movies, display the menu, accept user choices, select the chosen movie, and call that movie's booking and cancellation methods. |
| State (attributes) | A collection of the movies known to the system.                                                                                                   |

|                     |                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Behaviour (methods) | A method to display movies, a method to handle booking, a method to handle cancellation, and a main loop method that runs the menu until the user exits. |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5. Encapsulation Requirement

The booked-seat data must be **private** to the `Movie` class — for example, stored in an attribute such as `__booked_seats`. Code outside the class must not read or modify this data directly. Instead, the class provides a controlled accessor method (for example, `get_booked_seats()`) that returns the information in a read-only manner.

This is the core encapsulation lesson: the `BookingSystem` can ask a `Movie` what is booked, but it cannot reach in and change the seats itself. All changes flow through the movie's own booking and cancellation methods, which is where validation lives.

## 6. Sample Program Flow

The application presents a numbered menu and repeats until the user chooses to exit.

```
Movie Ticket Booking System

1. View Movies
2. View Seats
3. Book Ticket
4. Cancel Ticket
5. Booking Status
6. Exit

Enter choice:
```

A typical booking interaction:

```
Enter choice: 3

Select Movie: Leo

Enter Seat Number: A3

Ticket Booked Successfully
```

## 7. Constraints

To keep the focus on object-oriented design rather than data volume, every movie offers the same fixed set of five seats:

## 8. Implementation Approach

The following steps describe *how* to build the system using the object-oriented concepts above. They outline the design and reasoning; the actual code is left for the developer to write.

1. **Model the domain first.** Identify the two entities before writing anything: a single `Movie` and the `BookingSystem` that contains many movies. Decide what data each one truly owns. A movie owns its seats; the system owns the list of movies.
2. **Design the `Movie` constructor.** When a movie object is created, its `__init__` should accept the movie name and set up its seat state — a collection of available seats initialised to A1–A5, and an empty private collection for booked seats. This guarantees every movie starts in a valid, consistent state.
3. **Put behaviour where the data lives.** Because the `Movie` owns the seats, the logic that changes them belongs inside the `Movie` class. The booking method should validate that the seat exists and is currently free, then move it from available to booked. The cancellation method should verify the seat is currently booked, then return it to the available collection.
4. **Keep booked seats private.** Store booked seats in a private attribute and never expose it for direct modification. Provide a read-only accessor so other objects can view the bookings without being able to alter them. This is what makes the design encapsulated rather than just organised.
5. **Make `BookingSystem` the coordinator.** It holds the movies (a list works well, or a dictionary keyed by movie name for quick lookup) and exposes a menu loop. When the user books or cancels, the system selects the correct `Movie` object and calls that object's method — it must not touch the seat data itself. This demonstrates object interaction and respects encapsulation.
6. **Validate at every entry point.** Check that the chosen movie exists, that the entered seat is one of the valid seats, and that the requested operation makes sense (you cannot book a taken seat or cancel a free one). Return clear, user-friendly messages for each failure case.
7. **Drive everything from a menu loop.** A single run method should display the menu, read the user's choice, dispatch to the appropriate operation, and repeat until the user selects Exit. Keeping this loop thin — delegating real work to the class methods — keeps the program readable and maintainable.
8. **Test each path manually.** Confirm booking a free seat, rejecting a taken seat, cancelling a booked seat, rejecting cancellation of a free seat, and that status always reflects reality. Each successful path proves a different concept is working as intended.

Concept-to-feature mapping: **classes & objects** appear as `Movie` and `BookingSystem`; the **constructor** initialises seat state; **encapsulation** protects the booked seats; **object interaction** happens when the system delegates to a movie; **collections** store seats and movies; and **validation** plus the **menu loop** hold the application together.

## 9. Class Relationship

---

The `BookingSystem` contains and coordinates many `Movie` objects. Each movie manages its own seats internally.

```
BookingSystem
|
| contains
v
Movie
├─ movie_name
├─ available_seats
├─ __booked_seats (private)
├─ book_ticket()
├─ cancel_ticket()
└─ show_available_seats()
```

This structure is well suited to a developer who has covered classes and objects, constructors, lists, dictionaries, and encapsulation, and provides a compact but realistic exercise in applying those concepts together.